

CS 322

Camera Calibration & Stereo Vision



Image Credit: Pixar Animation Studios

CS 322

On the previous episode...

- Camera geometry



CS 322

On this episode...

- Camera calibration



Common Transformations



Original

Transformed



Translation



Rotation



Scaling



Affine



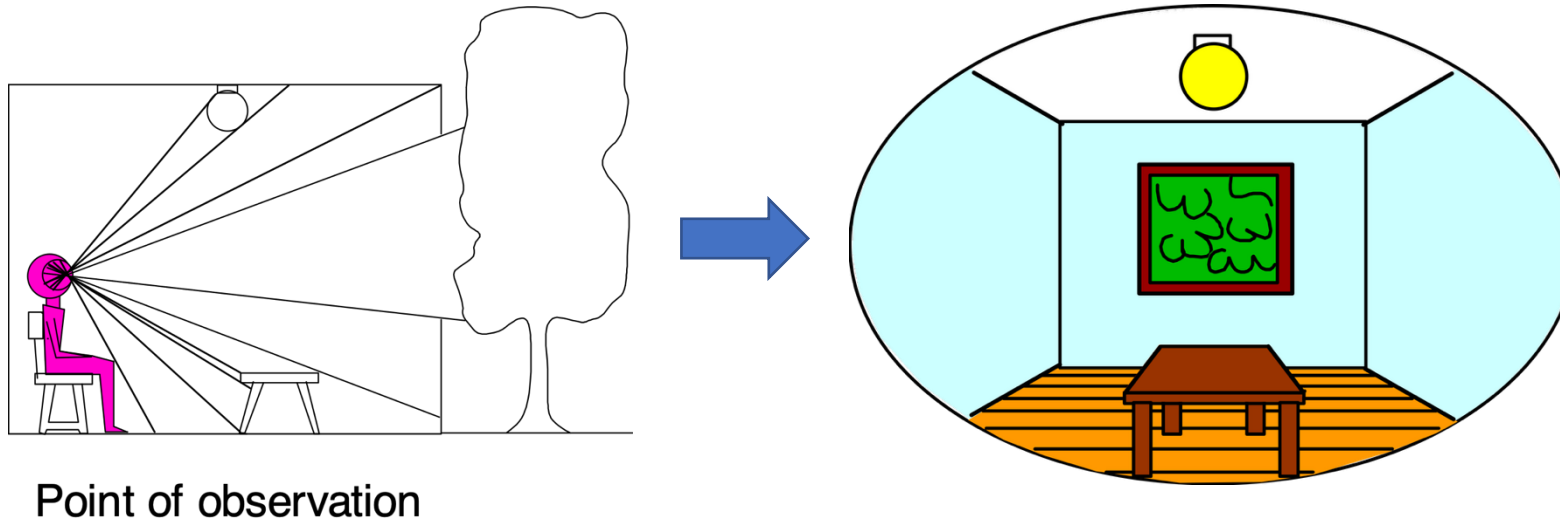
Perspective

Cameras are Projective Transformation Machines

Projective Transformation

3D world

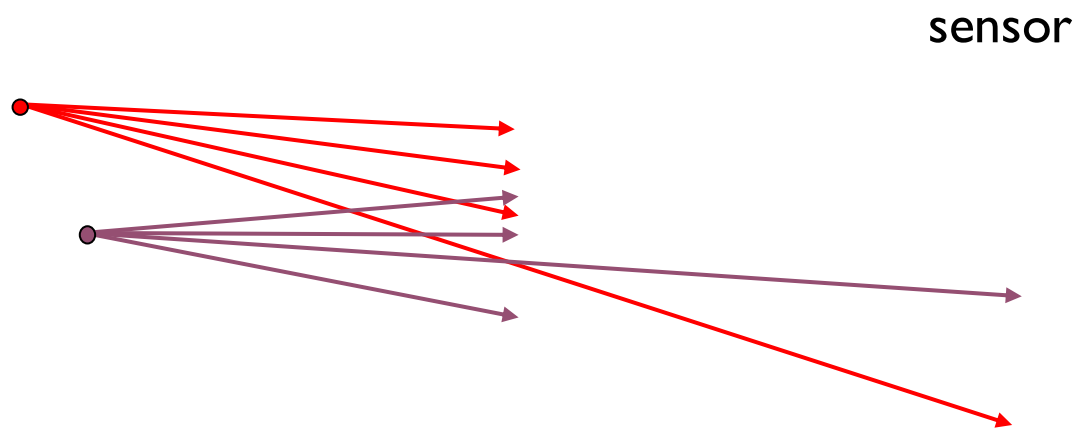
2D image



Design a Pinhole Camera

Idea 2: Add a barrier to block most rays

- Pinhole in barrier
- Only sense light from one direction.
 - Reduces blurring.
- In most cameras, this **aperture** can vary in size.



Fundamental Equations of Computer Vision

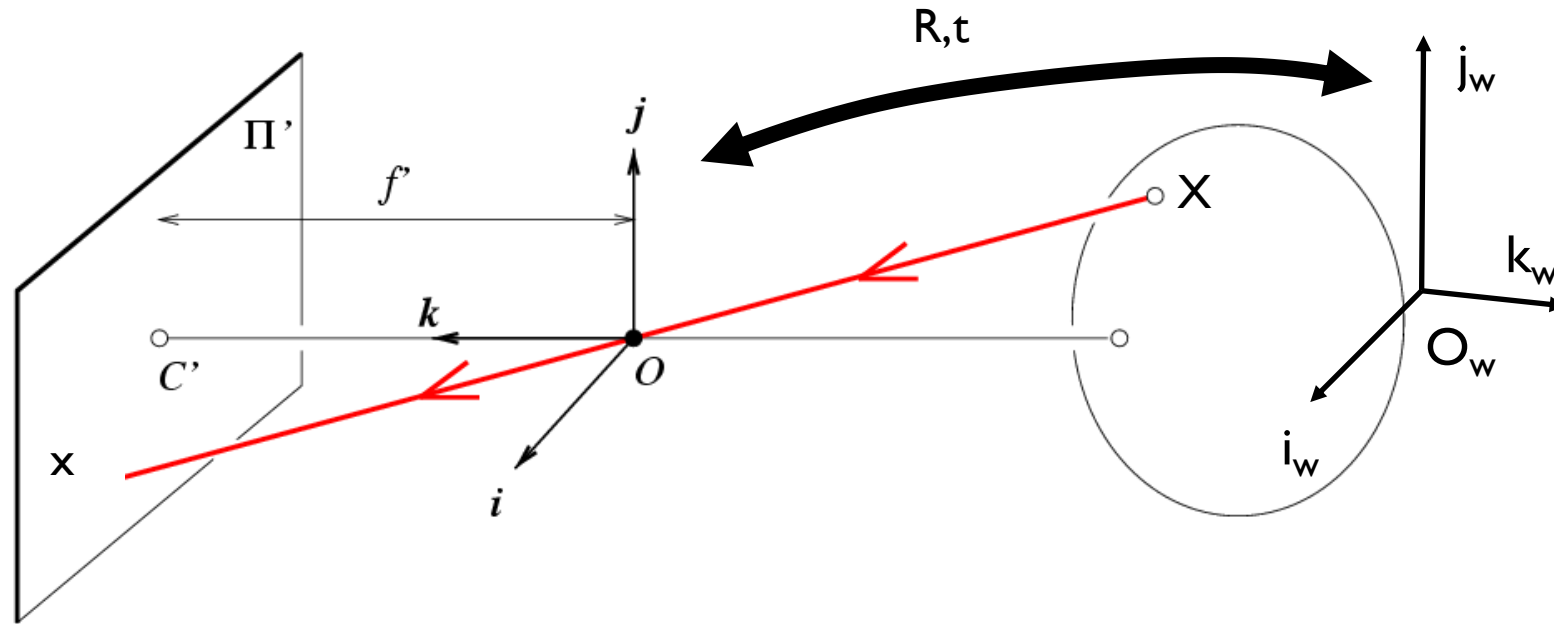
1. Image Filtering
$$h[m,n] = \sum_{k,l} f[k,l] I[m+k, n+l]$$

2. Camera Geometry
$$\mathbf{x} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \mathbf{X} \quad x^T F x' = 0$$

3. Machine Learning
$$\arg \min_{\mathbf{S}} \sum_{i=1}^k \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2. \quad y = \varphi\left(\sum_{i=1}^n w_i x_i + b\right) = \varphi(\mathbf{w}^T \mathbf{x} + b)$$

4. Optical Flow
$$I(x, y, t) = I(x + \Delta x, y + \Delta y, t + \Delta t)$$

Camera (Projection) Matrix



$$\mathbf{x} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \mathbf{X}$$

Extrinsic Matrix

$\mathbf{x}$: Image Coordinates: (u, v, l)

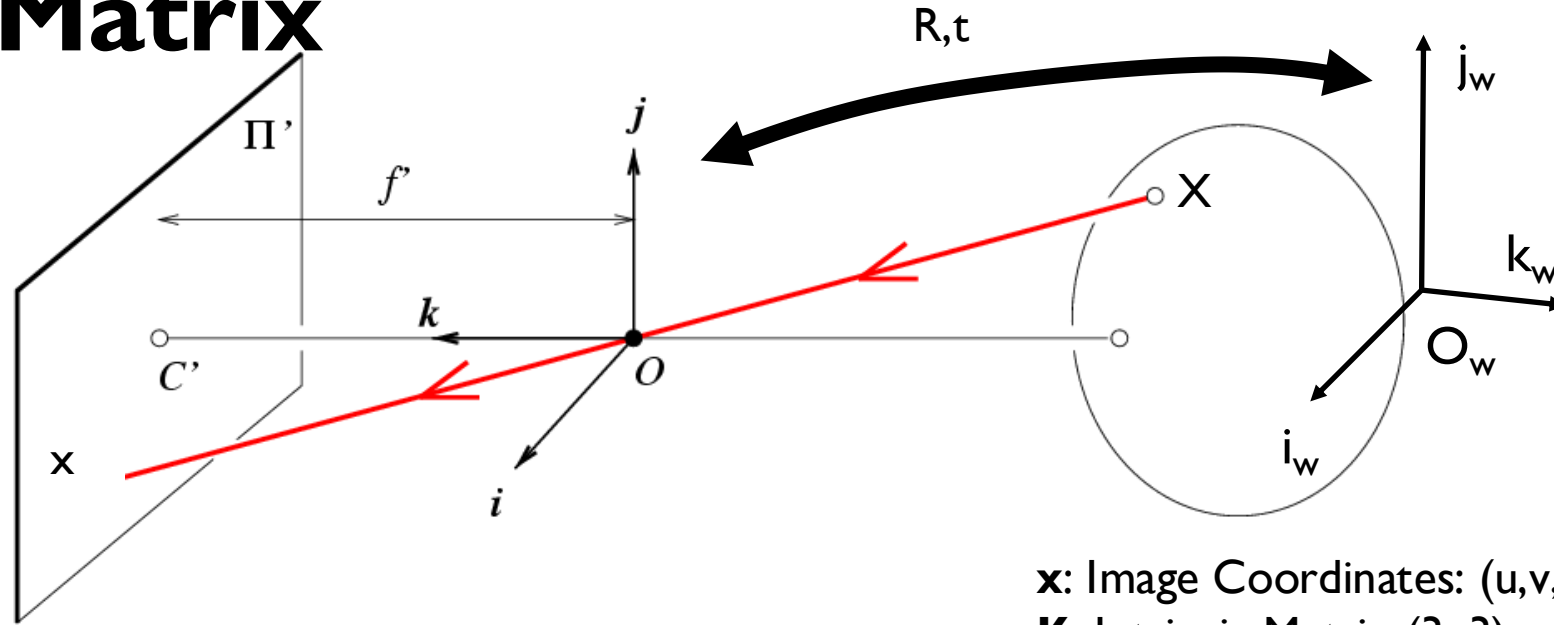
$\mathbf{K}$: Intrinsic Matrix (3×3)

$\mathbf{R}$: Rotation (3×3)

$\mathbf{t}$: Translation (3×1)

$\mathbf{X}$: World Coordinates: (X, Y, Z, l)

Camera Matrix



x: Image Coordinates: (u, v, l)

K: Intrinsic Matrix (3×3)

R: Rotation (3×3)

t: Translation (3×1)

X: World Coordinates: (X, Y, Z, l)

$$w \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & s & u_0 \\ 0 & f_y & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

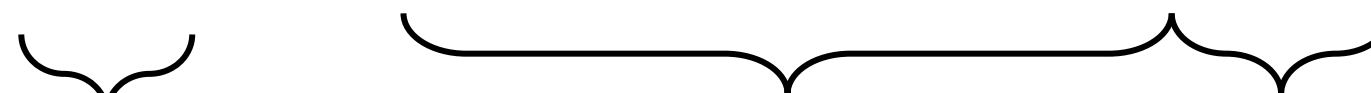
Simulate a Camera

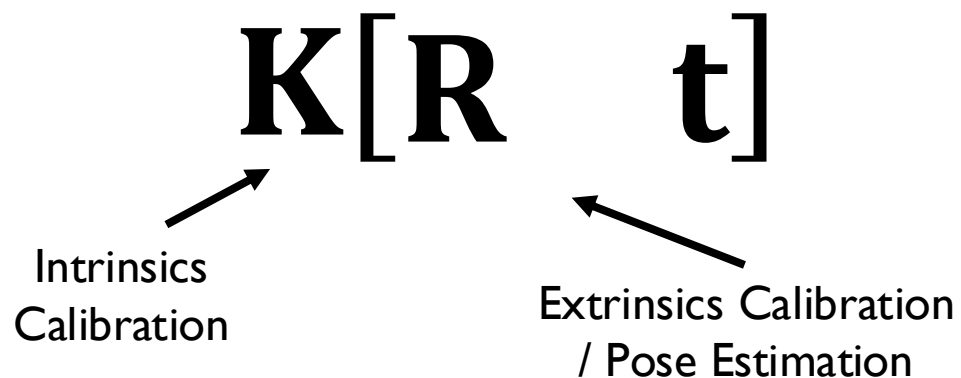
$$\mathbf{x} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \mathbf{X}$$


Unknown **Known** **Known**

Camera Calibration

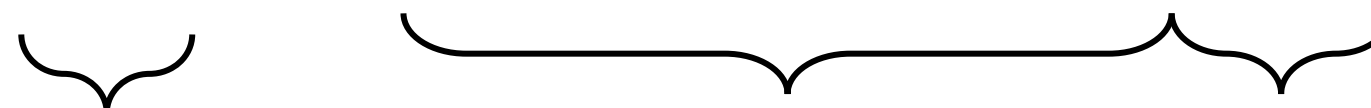
$$\mathbf{x} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \mathbf{X}$$


Known **Unknown** **Known**


K **[R t]**
Intrinsics Calibration Extrinsics Calibration
/ Pose Estimation

3D Reconstruction I

$$\mathbf{x} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \mathbf{X}$$


Known **Known** **Unknown**

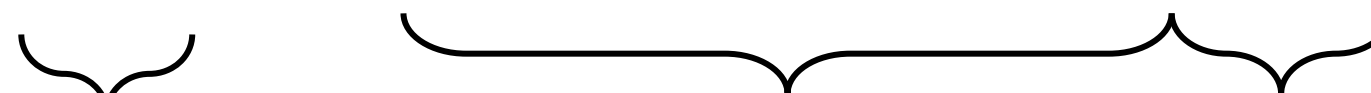
3D Reconstruction II

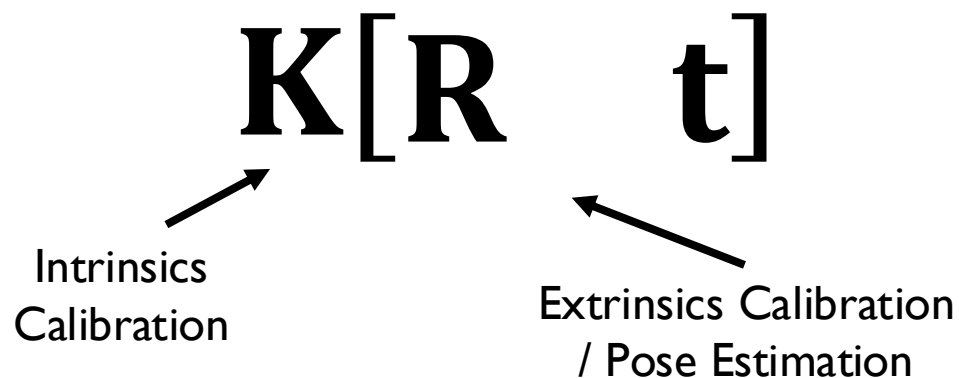
$$\mathbf{x} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \mathbf{X}$$


Known **Unknown Unknown**

Back to Camera Calibration

$$\mathbf{x} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \mathbf{X}$$


Known **Unknown** **Known**


K **[R t]**
Intrinsics Calibration Extrinsics Calibration
/ Pose Estimation

Camera Calibration / Resectioning

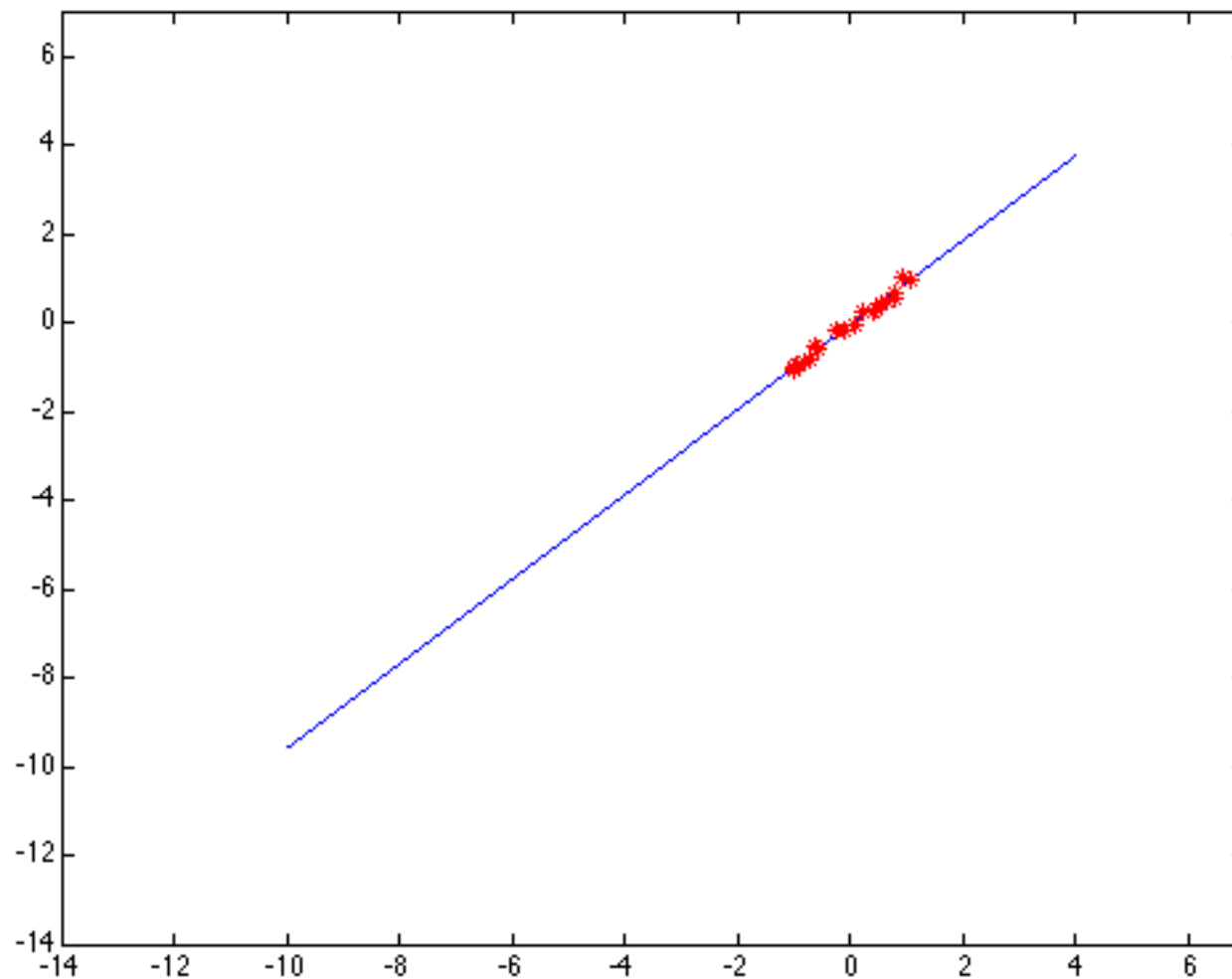
$$\mathbf{x} = \mathbf{K}[\mathbf{R} \quad \mathbf{t}] \mathbf{X}$$

$$\mathbf{x} = \mathbf{M}\mathbf{X}$$

$$\begin{bmatrix} wu \\ wv \\ w \end{bmatrix} = \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Linear least-squares regression!

Linear Regression: Fitting a line



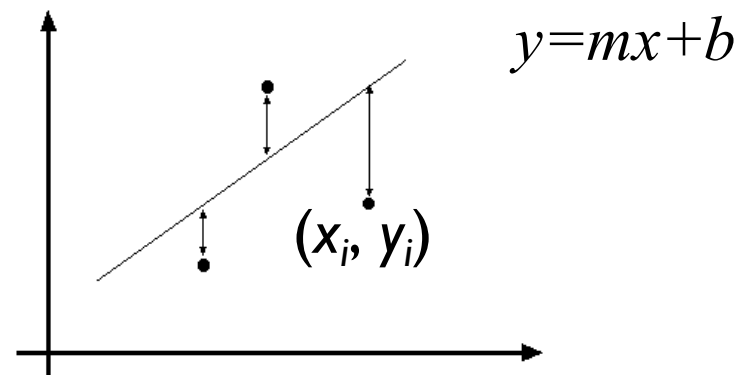
Linear Regression: Fitting a line

Data: $(x_1, y_1), \dots, (x_n, y_n)$

Line equation: $y_i = mx_i + b$

Find (m, b) to minimize

$$E = \sum_{i=1}^n (y_i - mx_i - b)^2$$



$$E = \sum_{i=1}^n \left([x_i \quad 1] \begin{bmatrix} m \\ b \end{bmatrix} - y_i \right)^2 = \left\| \begin{bmatrix} x_1 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix} \begin{bmatrix} m \\ b \end{bmatrix} - \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \right\|^2 = \|\mathbf{A}\mathbf{p} - \mathbf{y}\|^2$$

$$= \mathbf{y}^T \mathbf{y} - 2(\mathbf{A}\mathbf{p})^T \mathbf{y} + (\mathbf{A}\mathbf{p})^T (\mathbf{A}\mathbf{p})$$

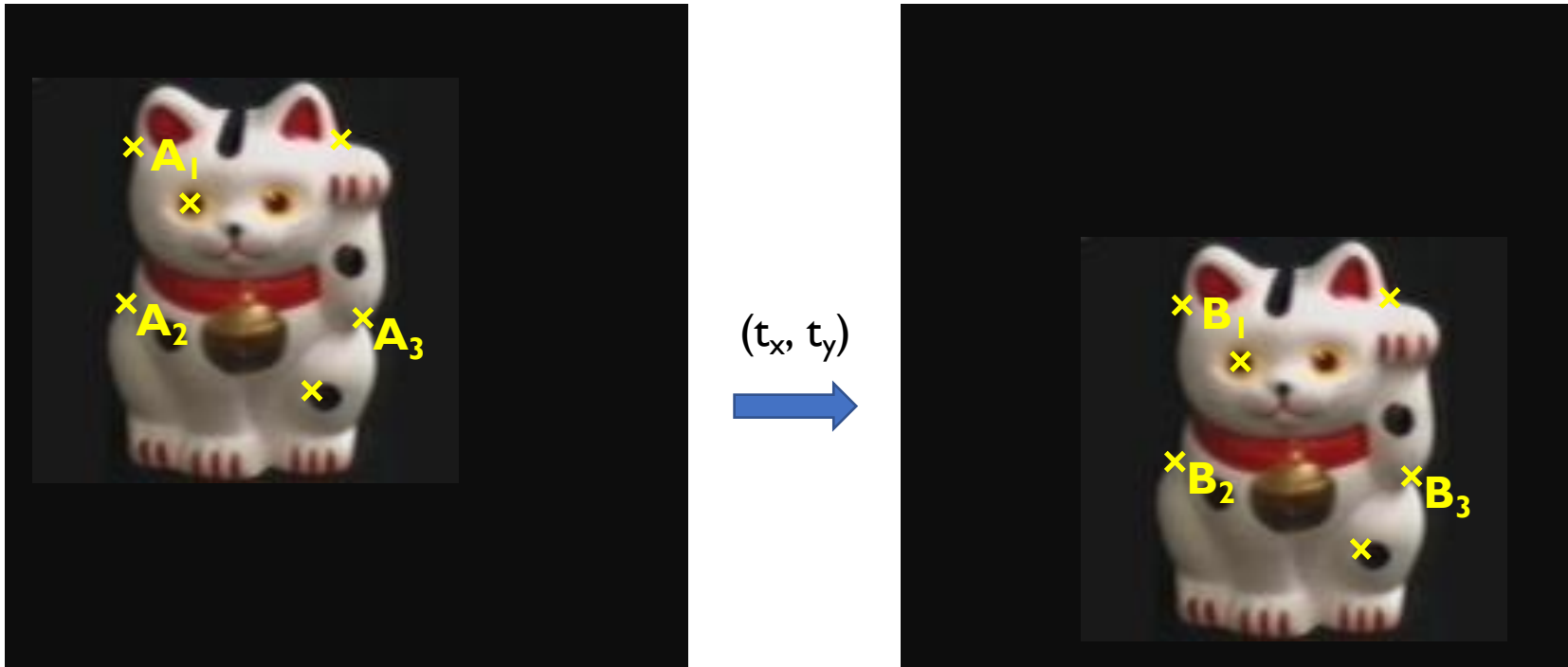
$$\frac{dE}{dp} = 2\mathbf{A}^T \mathbf{A}\mathbf{p} - 2\mathbf{A}^T \mathbf{y} = 0$$

Python:

```
p, residual = p.linalg.lstsq(A, y)
```

$$\mathbf{A}^T \mathbf{A}\mathbf{p} = \mathbf{A}^T \mathbf{y} \Rightarrow \mathbf{p} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{y} \quad (\text{Closed form solution})$$

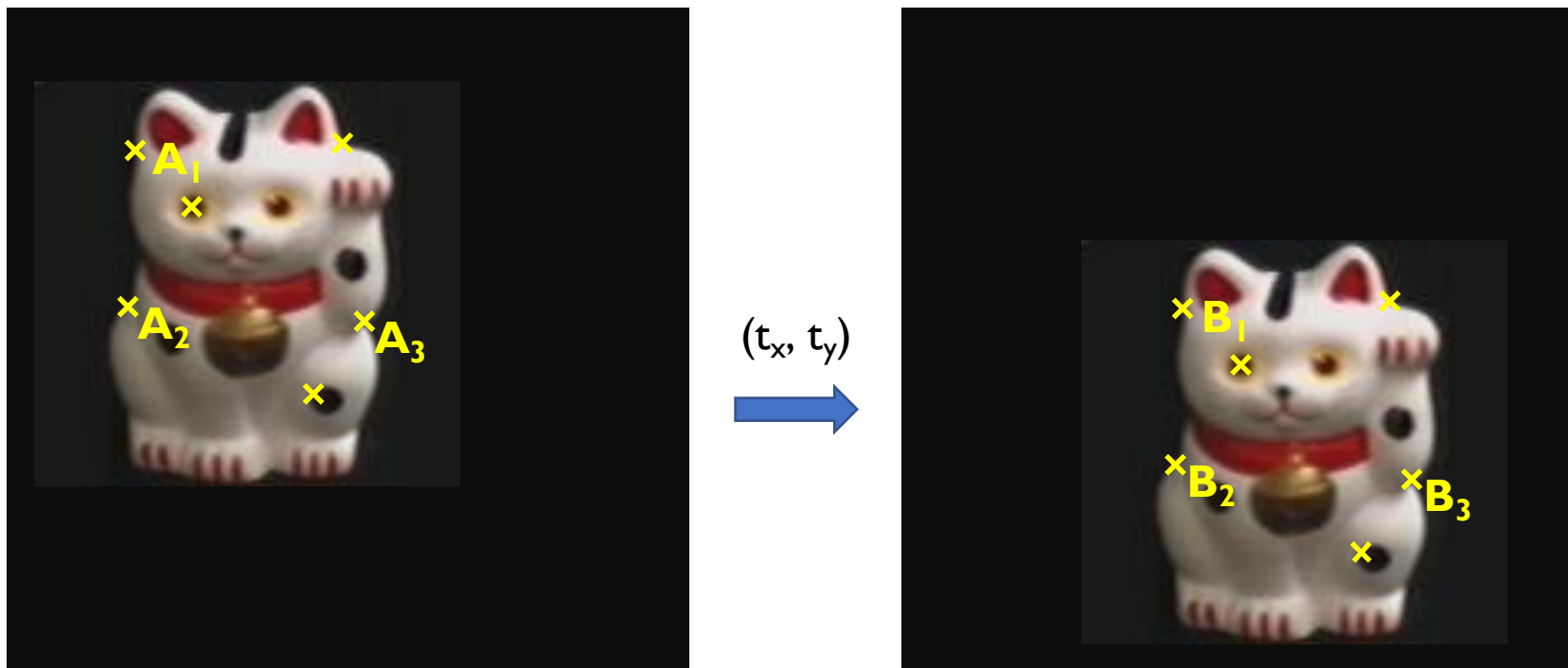
Example: Solving for Translation



$$\begin{bmatrix} x_i^B \\ y_i^B \end{bmatrix} = \begin{bmatrix} x_i^A \\ y_i^A \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

Given matched points in $\{A\}$ and $\{B\}$, estimate the translation of the object

Example: Solving for Translation

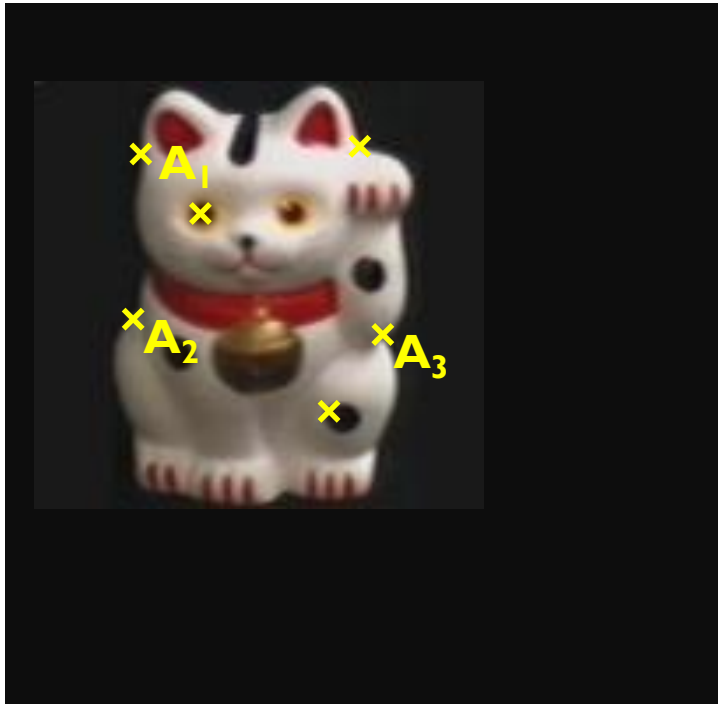


$$\begin{bmatrix} x_i^B \\ y_i^B \end{bmatrix} = \begin{bmatrix} x_i^A \\ y_i^A \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ \vdots & \vdots \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} t_x \\ t_y \end{bmatrix} = \begin{bmatrix} x_1^B - x_1^A \\ y_1^B - y_1^A \\ \vdots \\ x_n^B - x_n^A \\ y_n^B - y_n^A \end{bmatrix}$$

Least
squares
setup

Example: Solving for Rotation | Translation | Scale



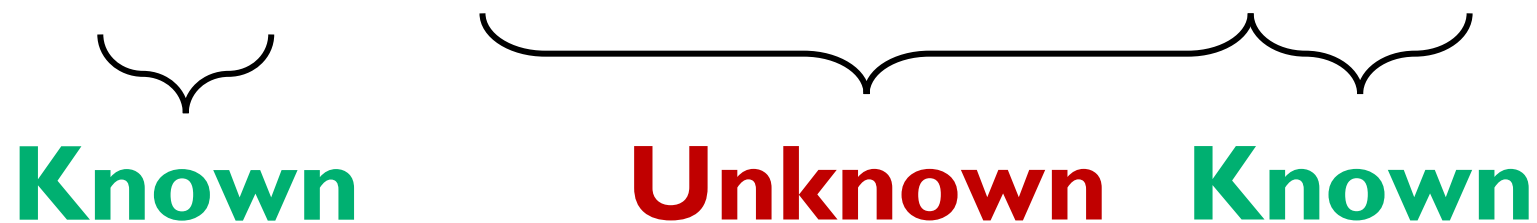
$$\begin{bmatrix} x_i^B \\ y_i^B \end{bmatrix} = \mathbf{T} \begin{bmatrix} x_i^A \\ y_i^A \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

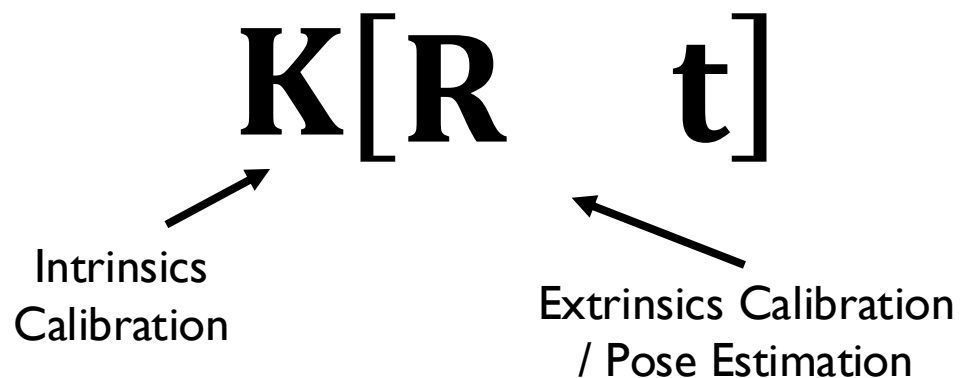
$$\mathbf{T} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

Given matched points in $\{A\}$ and $\{B\}$, estimate the translation of the object

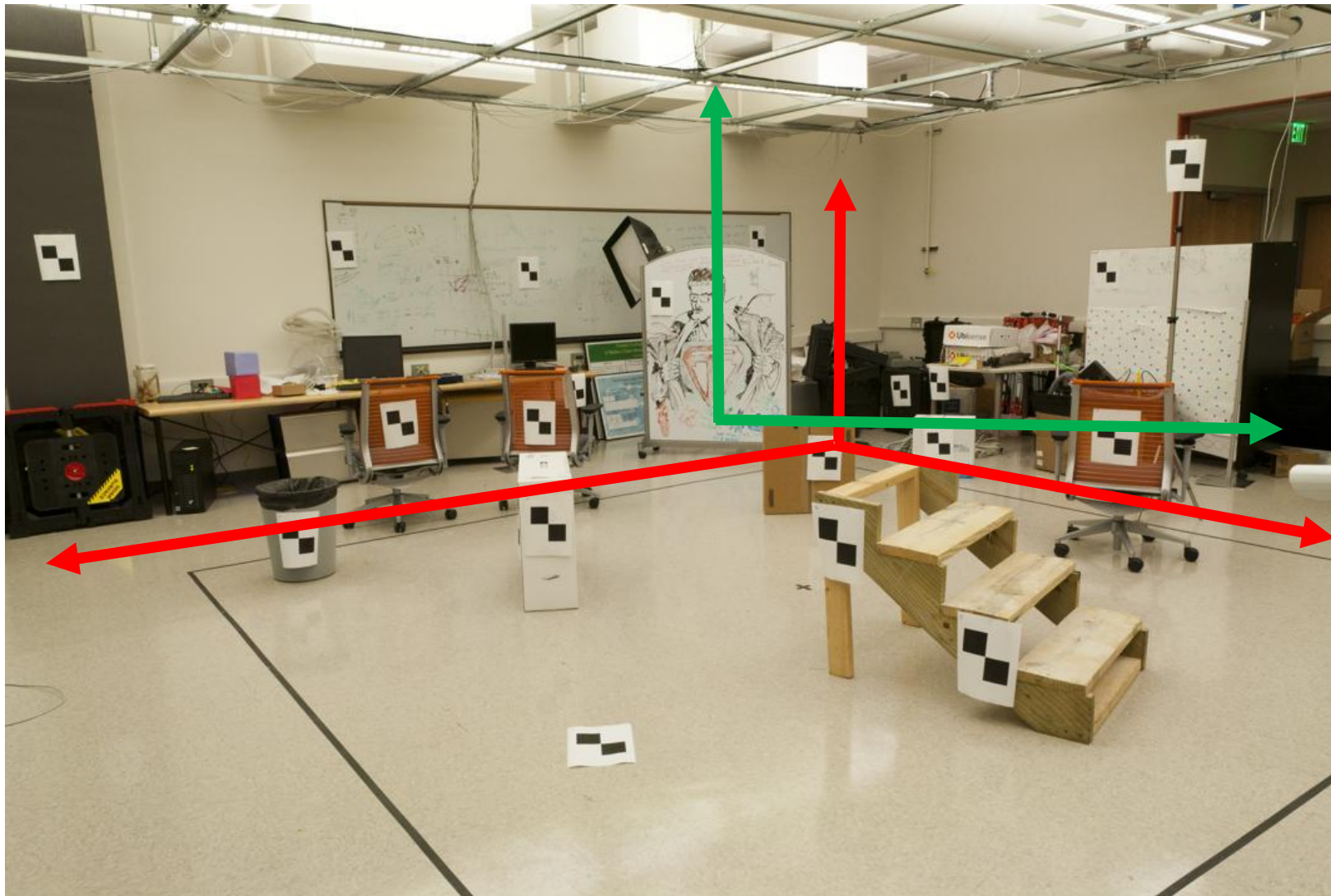
Camera Calibration

$$\mathbf{x} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \mathbf{X}$$


Known **Unknown** **Known**


K **[R t]**
Intrinsics Calibration Extrinsics Calibration
/ Pose Estimation

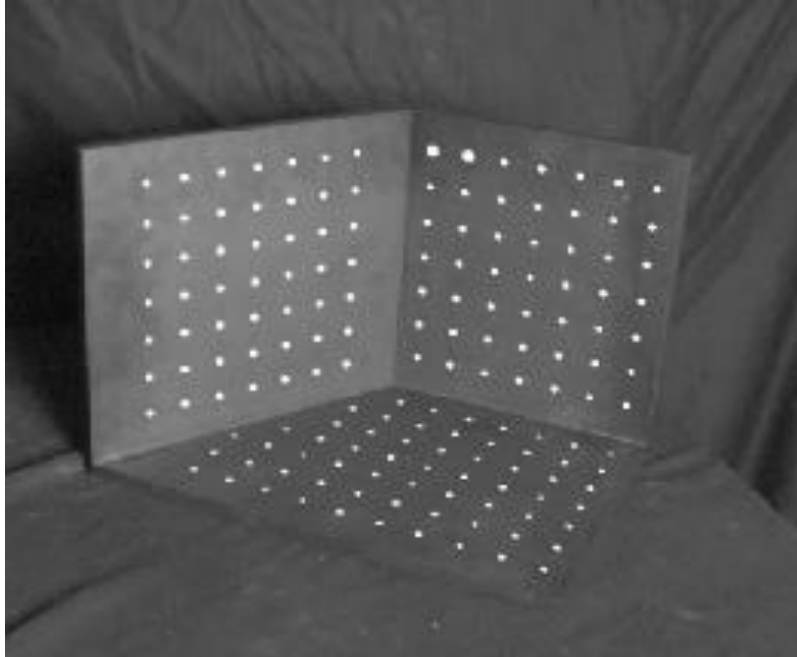
World vs Image Coordinates



Calibrating the Camera

Use a scene with **known** geometry

- Correspond image points to 3D points
- Get least squares solution (or non-linear solution)



Known 2D image
locations (px)



Known 3D world
locations (m)



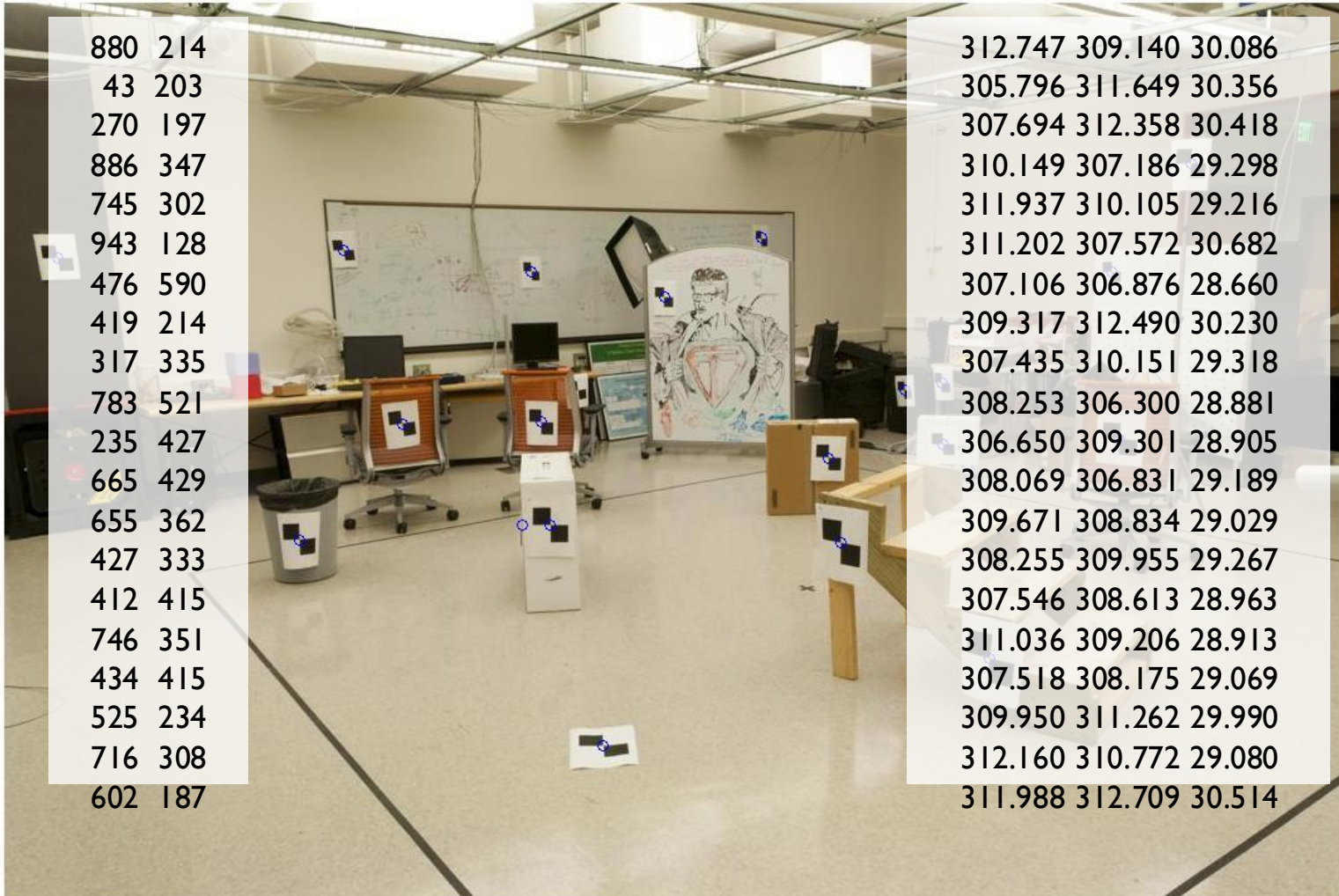
$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{matrix} \mathbf{M} \\ \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} \end{matrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$



Unknown Camera Parameters

Known 2D image coords

880 214
43 203
270 197
886 347
745 302
943 128
476 590
419 214
317 335
783 521
235 427
665 429
655 362
427 333
412 415
746 351
434 415
525 234
716 308
602 187

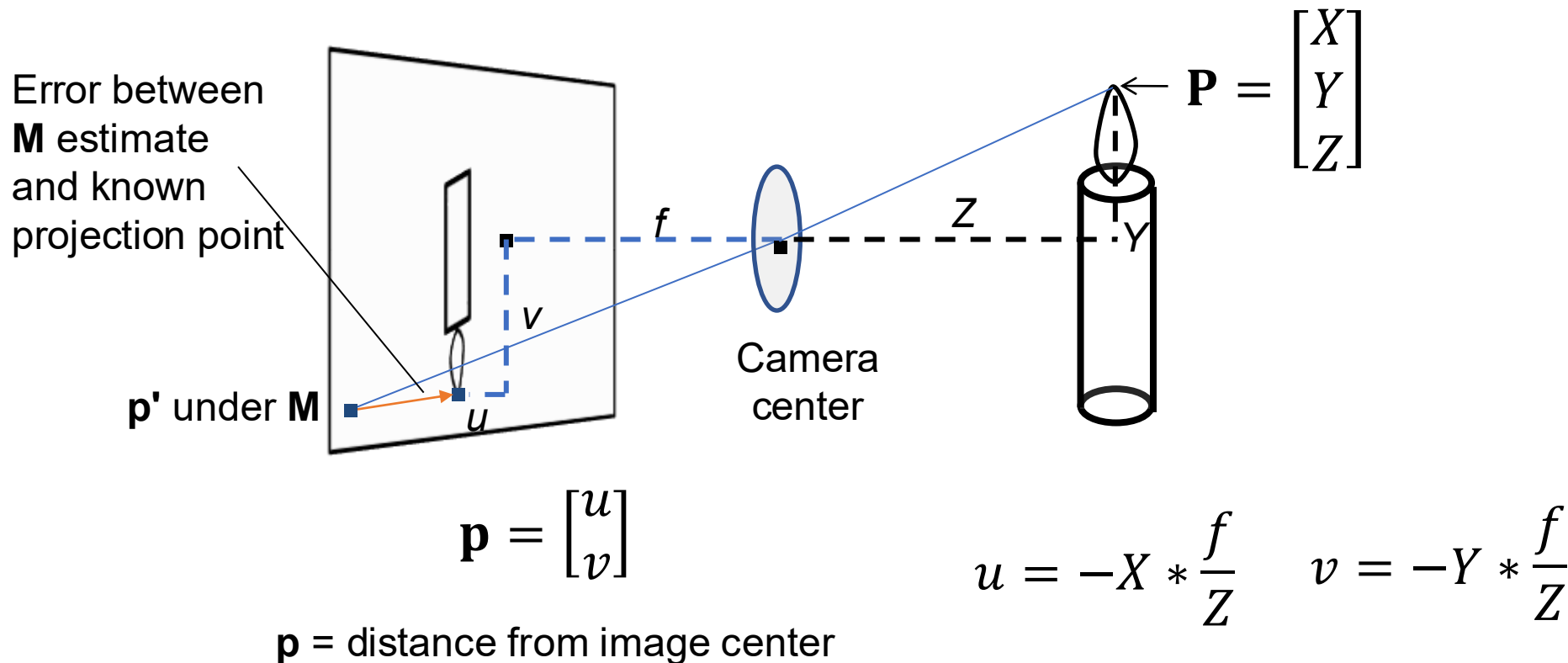


Known 3D world locations

312.747 309.140 30.086
305.796 311.649 30.356
307.694 312.358 30.418
310.149 307.186 29.298
311.937 310.105 29.216
311.202 307.572 30.682
307.106 306.876 28.660
309.317 312.490 30.230
307.435 310.151 29.318
308.253 306.300 28.881
306.650 309.301 28.905
308.069 306.831 29.189
309.671 308.834 29.029
308.255 309.955 29.267
307.546 308.613 28.963
311.036 309.206 28.913
307.518 308.175 29.069
309.950 311.262 29.990
312.160 310.772 29.080
311.988 312.709 30.514

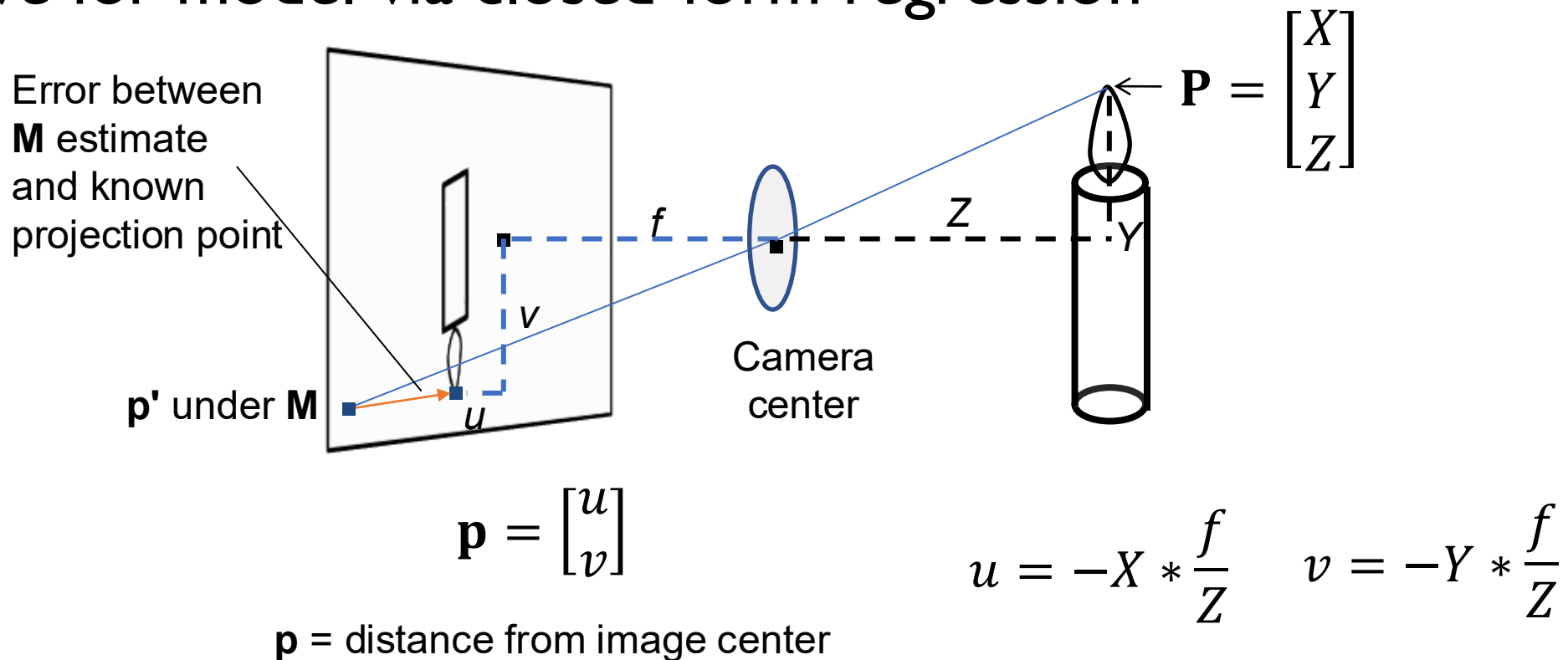
Least Squares Objective

Given 3D point evidence, find best **M** which minimizes error between estimate (**p'**) and known corresponding 2D points (**p**).



Least Squares Objective

- Best $\mathbf{M}$ occurs when $\mathbf{p}' = \mathbf{p}$, or when $\mathbf{p}' - \mathbf{p} = 0$
- Form these equations from all point evidence
- Solve for model via closed-form regression



Unknown Camera Parameters

Known 2d
image locations

$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Known 3d world
locations

First, work out
where X,Y,Z
projects to under
candidate **M**.

$$su = m_{11}X + m_{12}Y + m_{13}Z + m_{14}$$

$$sv = m_{21}X + m_{22}Y + m_{23}Z + m_{24}$$

$$s = m_{31}X + m_{32}Y + m_{33}Z + m_{34}$$

Two equations
per 3D point
correspondence

$$u = \frac{m_{11}X + m_{12}Y + m_{13}Z + m_{14}}{m_{31}X + m_{32}Y + m_{33}Z + m_{34}}$$

$$v = \frac{m_{21}X + m_{22}Y + m_{23}Z + m_{24}}{m_{31}X + m_{32}Y + m_{33}Z + m_{34}}$$

Unknown Camera Parameters



Known 2d
image locations

$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Known 3d world
locations

Next, rearrange into form
where all **M** coefficients are
individually stated in terms
of X,Y,Z,u,v.

-> Allows us to form lstsq
matrix.

$$u = \frac{m_{11}X + m_{12}Y + m_{13}Z + m_{14}}{m_{31}X + m_{32}Y + m_{33}Z + m_{34}}$$

$$v = \frac{m_{21}X + m_{22}Y + m_{23}Z + m_{24}}{m_{31}X + m_{32}Y + m_{33}Z + m_{34}}$$

$$(m_{31}X + m_{32}Y + m_{33}Z + m_{34})u = m_{11}X + m_{12}Y + m_{13}Z + m_{14}$$

$$(m_{31}X + m_{32}Y + m_{33}Z + m_{34})v = m_{21}X + m_{22}Y + m_{23}Z + m_{24}$$

$$m_{31}uX + m_{32}uY + m_{33}uZ + m_{34}u = m_{11}X + m_{12}Y + m_{13}Z + m_{14}$$

$$m_{31}vX + m_{32}vY + m_{33}vZ + m_{34}v = m_{21}X + m_{22}Y + m_{23}Z + m_{24}$$

Unknown Camera Parameters

↓

$$\begin{matrix} \text{Known 2d} \\ \text{image locations} \end{matrix} \begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \begin{matrix} \text{Known 3d world} \\ \text{locations} \end{matrix}$$

Next, rearrange into form
where all **M** coefficients are
individually stated in terms
of X,Y,Z,u,v.

-> Allows us to form lstsq
matrix.

$$m_{31}uX + m_{32}uY + m_{33}uZ + m_{34}u = m_{11}X + m_{12}Y + m_{13}Z + m_{14}$$

$$m_{31}vX + m_{32}vY + m_{33}vZ + m_{34}v = m_{21}X + m_{22}Y + m_{23}Z + m_{24}$$

$$0 = m_{11}X + m_{12}Y + m_{13}Z + m_{14} - m_{31}uX - m_{32}uY - m_{33}uZ - m_{34}u$$

$$0 = m_{21}X + m_{22}Y + m_{23}Z + m_{24} - m_{31}vX - m_{32}vY - m_{33}vZ - m_{34}v$$

Unknown Camera Parameters

↓

$$\begin{matrix} \text{Known 2d} \\ \text{image locations} \end{matrix} \begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \begin{matrix} \text{Known 3d world} \\ \text{locations} \end{matrix}$$

Finally, solve for m's entries using linear least squares

Method I – **Ax=b** form

$$\begin{bmatrix} X_1 & Y_1 & Z_1 & 1 & 0 & 0 & 0 & 0 & -u_1 X_1 & -u_1 Y_1 & -u_1 Z_1 \\ 0 & 0 & 0 & 0 & X_1 & Y_1 & Z_1 & 1 & -v_1 X_1 & -v_1 Y_1 & -v_1 Z_1 \\ & & & & & & \vdots & & & & \\ X_n & Y_n & Z_n & 1 & 0 & 0 & 0 & 0 & -u_n X_n & -u_n Y_n & -u_n Z_n \\ 0 & 0 & 0 & 0 & X_n & Y_n & Z_n & 1 & -v_n X_n & -v_n Y_n & -v_n Z_n \end{bmatrix} \begin{bmatrix} m_{11} \\ m_{12} \\ m_{13} \\ m_{14} \\ m_{21} \\ m_{22} \\ m_{23} \\ m_{24} \\ m_{31} \\ m_{32} \\ m_{33} \end{bmatrix} = m_{34} \begin{bmatrix} u_1 \\ v_1 \\ \vdots \\ u_n \\ v_n \end{bmatrix}$$

A **x** **b**

Python Numpy:

```
M, residual = np.linalg.lstsq(A,b)
```

```
M = np.append(M,1)
```

```
M = np.reshape(M, (3,4))
```

where camera matrix M is x.

Note: Must reshape M afterwards!

Unknown Camera Parameters

↓

$$\begin{matrix} \text{Known 2d} \\ \text{image locations} \end{matrix} \begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \begin{matrix} \text{Known 3d world} \\ \text{locations} \end{matrix}$$

Or, solve for m's entries using total linear least-squares

Method 2 – $\mathbf{Ax}=\mathbf{0}$ form

Find non-trivial solution (not $\mathbf{A}=\mathbf{0}$)

$$\mathbf{A} \mathbf{x} = \mathbf{0}$$

$$\begin{bmatrix} X_1 & Y_1 & Z_1 & 1 & 0 & 0 & 0 & 0 & -u_1 X_1 & -u_1 Y_1 & -u_1 Z_1 & -u_1 \\ 0 & 0 & 0 & 0 & X_1 & Y_1 & Z_1 & 1 & -v_1 X_1 & -v_1 Y_1 & -v_1 Z_1 & -v_1 \\ & & & & & \vdots & & & & & & \\ X_n & Y_n & Z_n & 1 & 0 & 0 & 0 & 0 & -u_n X_n & -u_n Y_n & -u_n Z_n & -u_n \\ 0 & 0 & 0 & 0 & X_n & Y_n & Z_n & 1 & -v_n X_n & -v_n Y_n & -v_n Z_n & -v_n \end{bmatrix} \begin{bmatrix} m_{11} \\ m_{12} \\ m_{13} \\ m_{14} \\ m_{21} \\ m_{22} \\ m_{23} \\ m_{24} \\ m_{31} \\ m_{32} \\ m_{33} \\ m_{34} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Python Numpy:

```
U, S, Vh = np.linalg.svd(a)
```

```
# V = Vh.T
```

```
M = Vh[-1, :]
```

```
M = np.reshape(M, (3,4))
```

Known 2D image coords

Known 3D world locations

1st point

880 214

 (u_1, v_1)

43 203

270 197

886 347

745 302

943 128

476 590

419 214

317.335

...

312.747 309.140 30.086 (X_1, Y_1, Z_1)

305.796 311.649 30.356

307.694 312.358 30.418

310.149 307.186 29.298

311.937 310.105 29.216

311.202 307.572 30.682

307.106 306.876 28.660

309.317 312.490 30.230

307.435 310.151 29.318

.....

Projection error defined by two equations – one for u and one for v

$$\begin{bmatrix}
 312.747 & 309.140 & 30.086 & 1 & 0 & 0 & 0 & 0 & -880 \times 312.747 & -880 \times 309.140 & -880 \times 30.086 & -880 \\
 0 & 0 & 0 & 0 & 312.747 & 309.140 & 30.086 & 1 & -214 \times 312.747 & -214 \times 309.140 & -214 \times 30.086 & -214 \\
 & & & & \vdots & & & & & & & \\
 X_n & Y_n & Z_n & 1 & 0 & 0 & 0 & 0 & -u_n X_n & -u_n Y_n & -u_n Z_n & -u_n \\
 0 & 0 & 0 & 0 & X_n & Y_n & Z_n & 1 & -v_n X_n & -v_n Y_n & -v_n Z_n & -v_n
 \end{bmatrix}
 \begin{bmatrix}
 m_{11} \\
 m_{12} \\
 m_{13} \\
 m_{14} \\
 m_{21} \\
 m_{22} \\
 m_{23} \\
 m_{24} \\
 m_{31} \\
 m_{32} \\
 m_{33} \\
 m_{34}
 \end{bmatrix}
 =
 \begin{bmatrix}
 0 \\
 0 \\
 \vdots \\
 0 \\
 0
 \end{bmatrix}$$

Known 2D image coords

Known 3D world locations

2nd point

 (u_2, v_2) (X_2, Y_2, Z_2)

880	214
43	203
270	197
886	347
745	302
943	128
476	590
419	214
317.	335

312.747	309.140	30.086
305.796	311.649	30.356
307.694	312.358	30.418
310.149	307.186	29.298
311.937	310.105	29.216
311.202	307.572	30.682
307.106	306.876	28.660
309.317	312.490	30.230
307.435	310.151	29.318

Projection error defined by two equations – one for u and one for v

$$\begin{bmatrix} 312.747 & 309.140 & 30.086 & 1 & 0 & 0 & 0 & 0 & -880 \times 312.747 & -880 \times 309.140 & -880 \times 30.086 & -880 \\ 0 & 0 & 0 & 0 & 312.747 & 309.140 & 30.086 & 1 & -214 \times 312.747 & -214 \times 309.140 & -214 \times 30.086 & -214 \\ 305.796 & 311.649 & 30.356 & 1 & 0 & 0 & 0 & 0 & -43 \times 305.796 & -43 \times 311.649 & -43 \times 30.356 & -43 \\ 0 & 0 & 0 & 0 & 305.796 & 311.649 & 30.356 & 1 & -203 \times 305.796 & -203 \times 311.649 & -203 \times 30.356 & -203 \\ & & & & \vdots & & & & & & & \\ X_n & Y_n & Z_n & 1_n & 0 & 0 & 0 & 0 & -u_n X_n & -u_n Y_n & -u_n Z_n & -u_n \\ 0 & 0 & 0 & 0 & X_n & Y_n & Z_n & 1 & -v_n X_n & -v_n Y_n & -v_n Z_n & -v_n \end{bmatrix} \begin{bmatrix} m_{11} \\ m_{12} \\ m_{13} \\ m_{14} \\ m_{21} \\ m_{22} \\ m_{23} \\ m_{24} \\ m_{31} \\ m_{32} \\ m_{33} \\ m_{34} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

How many points needed to fit the model?

$$\mathbf{x} = \mathbf{K}[\mathbf{R} \quad \mathbf{t}] \mathbf{X}$$



Degrees of freedom?

$$w \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{matrix} 5 \\ \begin{bmatrix} \alpha & s & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix} \end{matrix} \begin{matrix} 6 \\ \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix} \end{matrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

M is 3x4, so 12 unknowns, but projective scale ambiguity – 11 deg. freedom.

One equation per unknown -> 5 1/2 point correspondences determines a solution (e.g., either u or v).

More than 5 1/2 point correspondences -> overdetermined, many solutions to **M**.

Least squares is finding the solution that best satisfies the overdetermined system.

Why use more than 6 points? Robustness to error in feature points.

Can we factorize $\mathbf{M}$ back to $\mathbf{K} [\mathbf{R} \mid \mathbf{T}]$?

- Yes! We can directly solve for the individual entries of $\mathbf{K} [\mathbf{R} \mid \mathbf{T}]$.
- See next three hidden slides

Factorize M back to $K [R \mid T]$

- We can also use RQ factorization (not QR)
 - R in RQ is not rotation matrix R ; crossed names!

R (upper triangular or 'Right' triangular) is K

Q (orthogonal basis) is R

T , the last column of $[R \mid T]$, is $\text{inv}(K) * \text{last column of } M$.

- But you need to do a bit of post-processing to make sure that the matrices are valid. See <http://ksimek.github.io/2012/08/14/decompose/>

Recovering the camera center

$$\mathbf{x} = \mathbf{K}[\mathbf{R} \quad \mathbf{t}] \mathbf{X}$$

This is not the camera center $\mathbf{C}$.

It is $-\mathbf{RC}$, as the point is rotated before t_x , t_y , and t_z are added

So we need $-\mathbf{R}^{-1} \mathbf{K}^{-1} \mathbf{m}_4$ to get $\mathbf{C}$.

$$w \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} \alpha & s & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} wu \\ wv \\ w \end{bmatrix} = \underbrace{\begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \end{bmatrix}}_{\mathbf{Q}} \underbrace{\begin{bmatrix} * \\ * \\ * \end{bmatrix}}_{\mathbf{m}_4} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

This is $\mathbf{t} \times \mathbf{K}$
So $\mathbf{K}^{-1} \mathbf{m}_4$ is $\mathbf{t}$

$\mathbf{Q}$ is $\mathbf{K} \times \mathbf{R}$.
So we just need $-\mathbf{Q}^{-1} \mathbf{m}_4$

Great! So now I have K and R_t

Marker-based augmented reality.

- Define marker with known real-world geometry
- Find points on marker in image
- Estimate camera pose from corresponding 2D/3D points
- Render graphics from virtual camera with same pose



Marker-based augmented reality



Markerless augmented reality – unknown scene geometry
“Holoportation”, Microsoft Research

Intermission / Questions

